

Industrial Vision

Lecture 3 - Edge & Line Detection

The gradient, Canny, and the Hough transform

What is an edge, and where does it come from?

An **edge** is a pixel at which the intensity changes sharply. It is not yet a line, since the grouping of edge pixels into lines is treated later in this lecture.

Why edges matter. A bare contour is often sufficient to recognize a table or a face. Edges are compact and highly informative low-level features, and, as the first layers of convolutional networks show, they are what a visual system extracts first.

An edge may have several **physical causes**.

- A discontinuity in **depth**, where one object lies in front of another.
- A change in the **surface normal**, such as a crease in the shape.
- A **shadow** or other illumination boundary.
- A change in **texture, color** or **reflectance**.

A depth discontinuity usually produces an edge, but an edge does not require one, as a painted line on a flat wall demonstrates. Edges are also the basis for locating **vanishing points** in the single-view geometry.

The edge gradient: magnitude and orientation

For an image $G(x, y)$ the gradient components are the partial derivatives, computed with the difference filters of L01:

$$G_x = \frac{\partial G}{\partial x}, \quad G_y = \frac{\partial G}{\partial y}$$

The **magnitude** measures how strong the edge is at that location.

$$\|\nabla G\| = \sqrt{G_x^2 + G_y^2}$$

The **orientation** indicates the direction in which the intensity increases.

$$\theta = \tan^{-1} \left(\frac{G_y}{G_x} \right)$$

It points **in the direction of steepest increase**.

A vertical edge running from 0 to 255 has $G_y = 0$ and therefore $\theta = 0^\circ$, whereas the reverse edge from 255 to 0 gives $\theta = 180^\circ$. Since `filter2D` computes correlation rather than convolution, and so does not flip the kernel, the vertical filter must be written as $\begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$ in order that an edge that is bright on the right is reported at $\theta = 0$.

The derivative of Gaussian

Recall from last lecture. For any linear filter w acting on $g = f + \eta$ with η i.i.d., the output noise variance is $\sigma_\eta^2 \sum_k w_k^2$. The gradient's $[1 \ 0 \ -1]$ has $\sum w^2 = 2$, so **differencing doubles the noise variance**; the Laplacian's $\sum w^2 = 20$ is why we couldn't use it raw.

The remedy, established there, is to smooth before differentiating. The two steps combine into a single operator, the **derivative of Gaussian**:

$$\frac{\partial}{\partial x} (G_\sigma * I) = \left(\frac{\partial G_\sigma}{\partial x} \right) * I$$

This is the *derivative* of a Gaussian, a first derivative used for gradients, and it should not be confused with the *difference* of Gaussians.

Sub-pixel location of a zero-crossing

The edge reported by the Laplacian lies *between* pixels, at the point where f'' **changes sign**. It is located by **linear interpolation**.

Suppose that $f'' = -6$ at one pixel, where $\delta=0$, and $+3$ at the next, where $\delta=1$. Interpolating gives

$$a(\delta) = (1 - \delta)(-6) + \delta(3)$$

Setting $a(\delta) = 0$ gives

$$-6 + 9\delta = 0 \Rightarrow \delta = \frac{2}{3}$$

- The crossing lies $\frac{2}{3}$ of the way toward the sample of smaller magnitude, that is, closer to $+3$, as expected.
- This is **sub-pixel localization**, since the edge does not coincide with a pixel.
- The same interpolation is used within the non-maximum suppression stage of Canny, which is described next.

Why thresholding the gradient magnitude fails

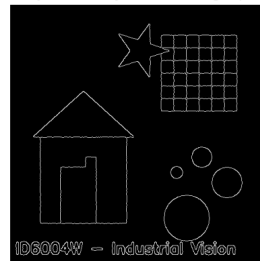
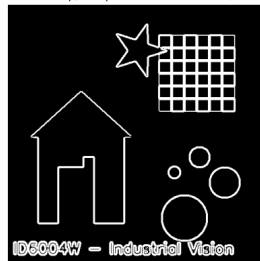
The obvious rule is to declare an edge at any pixel whose gradient magnitude $\|\nabla G\|$ exceeds a threshold. This rule fails on all three requirements of a good edge detector.

Good detection Real edges are found and few false ones are reported, the balance being set by the noise level.

Good localization The edge is reported at the correct position.

Single response Exactly **one** pixel is reported per edge.

threshold $|\text{grad}|$ (thick, multi-response) Canny: NMS + hysteresis (single-pixel)



A plain threshold produces a broad **band** around each edge, because the magnitude rises and falls over many pixels. This violates the requirement of **single response**, and only the local maximum should be retained.

The Canny detector: overview

In 1986 John Canny formulated edge detection as the **optimization** of those three criteria under Gaussian noise, and showed that the optimal filter is essentially a **derivative of Gaussian**.

The method has four steps.

1. **Smooth** the image with a Gaussian G_σ , which controls the noise and in which σ sets the scale.
2. Compute the **gradient** magnitude $\|\nabla G\|$ and orientation θ , using a derivative of Gaussian.
3. Apply **non-maximum suppression**, which thins each ridge to a single pixel.
4. Apply **hysteresis thresholding**, which links weak edges to strong ones.

Unlike the Laplacian, the gradient carries an **orientation**, and both steps 3 and 4 make use of θ .

Canny, step 3: non-maximum suppression

A pixel is retained **only if it is a local maximum of $\|\nabla G\|$ along the gradient direction.**

At a pixel a we examine the neighbors b and c lying **forward and backward along θ** :

$$\text{keep } a \iff \|\nabla G\|_a \geq \|\nabla G\|_b \text{ and } \geq \|\nabla G\|_c$$

If the condition does not hold, the pixel is set to 0, that is, it is suppressed.

Non-maximum suppression is a general technique. It is used again for detection boxes, for corner peaks in L05, and for Hough peaks later in this lecture.

The gradient direction seldom passes exactly through a grid point, so the values at b and c are **interpolated** from the two pixels that straddle the direction. This is the same linear interpolation used for the zero-crossing above.

The broad ridge is thereby reduced to a contour one pixel wide, which satisfies the requirement of **single response**.

Canny, step 4: hysteresis thresholding

Two thresholds $T_1 < T_2$ allow **weak but genuine** edges to be retained without admitting noise.

- A pixel with $\|\nabla G\| > T_2$ is a **strong** edge pixel and is retained.
- A pixel with $\|\nabla G\| < T_1$ is **not** an edge and is discarded. A common choice is $T_1 \approx T_2/2$.
- A pixel with $T_1 \leq \|\nabla G\| \leq T_2$ is **weak**, and is retained **only if** it is **8-connected** to a strong pixel through a chain of pixels that are themselves weak or stronger.

The hysteresis consists in the fact that a weak pixel is retained by virtue of its **connection** to a strong one. Natural edges are continuous, so a weak segment that bridges two strong ones is almost certainly genuine. This step prevents faint edges from becoming fragmented.

From edge pixels to lines

The Canny detector returns a set of edge pixels, many of which lie **on a common line**. It is preferable to report the line itself rather than the individual pixels, since a line is more informative and is robust to gaps.

A first attempt is to fit $y = mx + c$ and to vote for the parameters (m, c) . This fails because a vertical line has **infinite slope** m , which is unbounded and cannot be represented in a finite accumulator. Vertical lines are common in man-made scenes and cannot be ignored.

The solution is the **normal form** of a line, whose parameters are always finite:

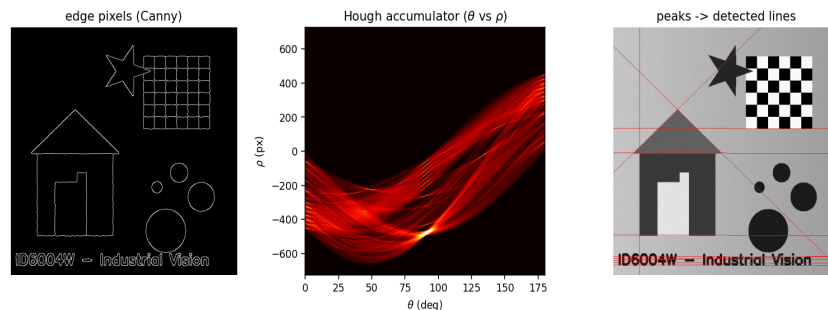
$$\rho = x \cos \theta + y \sin \theta$$

Here ρ is the perpendicular distance of the line from the origin and θ is the angle of that perpendicular. The vertical line $x = 5$ is represented simply by $\rho = 5$ and $\theta = 0$, and no infinite quantity arises.

The Hough transform: voting in (ρ, θ) space

The construction rests on a **duality**. If a point (x, y) is fixed and θ is varied, then $\rho(\theta) = x \cos \theta + y \sin \theta$ traces a **sinusoid** in (ρ, θ) space.

- Each **edge pixel** gives rise to one **sinusoid**.
- Points lying on the **same line** share a single pair (ρ, θ) , so their sinusoids all **intersect at that bin**.
- In the **voting** stage every pixel increments all the (ρ, θ) bins along its sinusoid, and a line appears as a **peak** with many votes.



For an $M \times M$ image the ranges are $\theta \in [0, \pi)$ and $\rho \in [-M/\sqrt{2}, M/\sqrt{2}]$, the latter being set by the longest diagonal.

The duality, on synthetic data

1 point

2 points

5 collinear points

2 lines

3 noisy lines + clutter

clear · click to add

sweep θ



35°

sweep θ

min votes



2

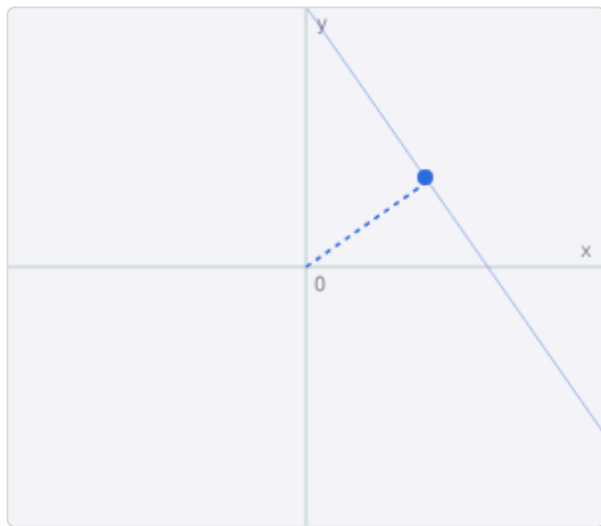
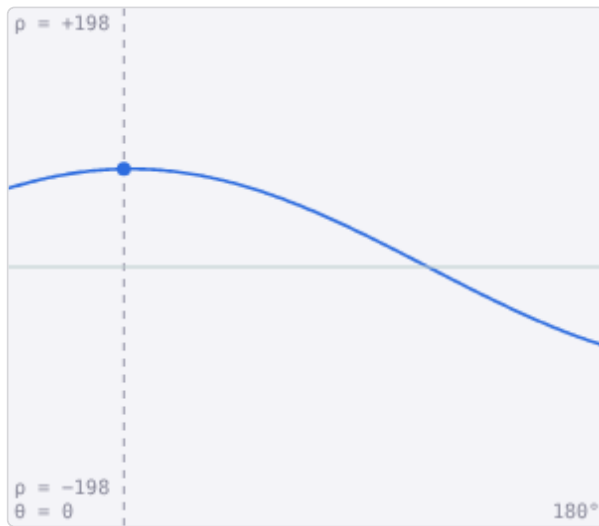


image space (x, y) — origin at the centre



Hough space (θ, ρ) , $\rho = x \cos \theta + y \sin \theta$

one point $\rightarrow$ one sinusoid; no single line is determined, so no peak

A single **point** traces a **sinusoid**: sweeping θ rotates the line through that point, and ρ is the signed distance of the origin from it. Points of a common **line** share one pair (ρ, θ) , so their sinusoids **cross at a single bin**. With many edge pixels the crossings become **peaks** in the accumulator, one per line.

The Hough transform: algorithm and trade-offs

Algorithm

1. Construct a quantized accumulator over (ρ, θ) .
2. For each **edge pixel** (x_i, y_i) , **increment** every bin lying on its sinusoid $\rho = x_i \cos \theta + y_i \sin \theta$.
3. Locate the **local maxima**, applying non-maximum suppression to the accumulator so that nearby duplicates are merged.
4. Report the pair (ρ, θ) at each peak as one detected line.

Strengths

- The method is conceptually simple and robust to noise.
- It **tolerates occlusion and gaps**, since the votes survive the loss of some pixels.
- It generalizes to circles and ellipses, at the cost of a higher-dimensional accumulator.

Limitations

- Only one type of primitive can be detected at a time, and the cost grows with the number of parameters.
- It returns an infinite line and therefore gives **neither endpoints nor length**.
- It may bridge a genuine gap, merging collinear segments that are in fact distinct.

Inline demonstration

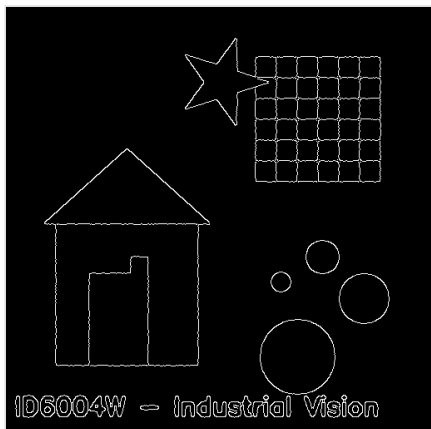
image Synthetic scene  *ideal step edges, exact corners — the textbook case*

blur σ 1.0 

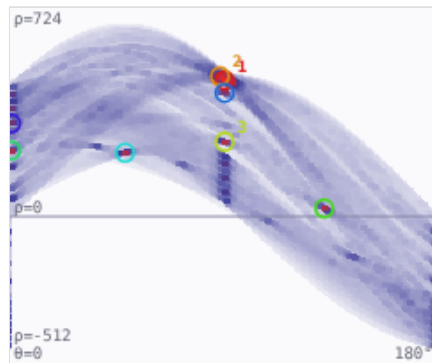
Canny high 150 

Canny low 60 

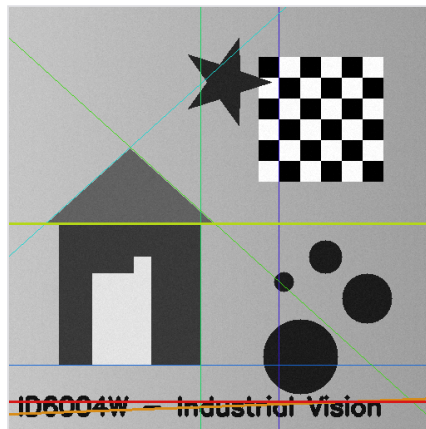
top-k lines 8 



Canny edges



Hough space (θ , ρ) — peaks ringed



top 8 lines, by votes

6901 edge pixels · top 8 peaks: 231 → 162 votes

Adjusting the blur σ and the Canny thresholds changes the thickness and number of edges. The middle panel is the **accumulator** itself: every edge pixel has voted along its sinusoid, and each ringed spot is a local maximum. The lines are then the **k highest peaks, sorted by votes**, coloured from red (most votes) to blue (fewest), so raising k adds progressively weaker lines.

Questions

- Q. Why is it not sufficient to threshold the gradient magnitude?
- Q. What does the orientation θ provide that the Laplacian did not?
- Q. Why is the **normal form** used in place of $y = mx + c$?
- Q. Can the Hough transform report where a line **begins and ends**?

Summary

- An **edge** is a sharp change of intensity, arising from depth, shape, shadow, texture or reflectance. The **gradient** gives its **strength** $\sqrt{G_x^2 + G_y^2}$ and its **orientation** $\theta = \tan^{-1}(G_y/G_x)$.
- Differencing **doubles** the noise variance, from σ^2 to $2\sigma^2$ (L02), and a **derivative of Gaussian** should therefore be used.
- The **Canny** detector smooths the image, computes the gradient, applies **non-maximum suppression** to obtain single-pixel edges, and applies **hysteresis** to link weak edges to strong ones. It thereby achieves good detection, good localization and single response.
- The **Hough transform** groups edge pixels into **lines** by voting in (ρ, θ) space, the normal form avoiding the difficulty of infinite slope. Peaks in the accumulator correspond to lines. The method is robust to gaps but gives no information about endpoints.

In the next lecture we observe that although edges and lines carry structure, an individual edge pixel **cannot be matched** between views, since moving along the edge produces no change. We therefore seek points that are unambiguous in every direction, namely **features, or corners**, and we study the **Harris** detector.

References

1. Forsyth, David A., and Jean Ponce. Industrial Vision: a modern approach. prentice hall professional technical reference, 2002. See **Ch. 5** on edge detection.
2. Canny, John. "A computational approach to edge detection." IEEE Transactions on pattern analysis and machine intelligence 6 (1986): 679-698.
3. Duda, Richard O., and Peter E. Hart. "Use of the Hough transformation to detect lines and curves in pictures." Communications of the ACM 15.1 (1972): 11-15.

Demos: `lec03_edges_lines.py`